

Benchmark entre PostgreSQL, MySQL y Python

Bases de Datos I

Estudiante: Samuel Palomino Restrepo
Profesor: Francisco Moreno
Curso: Bases de Datos I
Fecha: 30 de julio de 2026

Índice

1. Introducción	2
2. Esquema utilizado	2
3. Pruebas de carga	3
3.1. Resultados	4
4. Benchmark de consultas	4
4.1. Consulta 1	4
4.2. Consulta 2	5
5. Conclusiones	6

1. Introducción

El presente trabajo aborda la evaluación experimental del rendimiento entre dos de los sistemas de gestión de bases de datos relacionales (SGBD) más utilizados en la industria: PostgreSQL y MySQL. La velocidad de carga masiva de datos y el tiempo de respuesta en la ejecución de consultas complejas son factores críticos para determinar la escalabilidad de un sistema. Este benchmark compara ambas bases de datos ejecutadas en entornos aislados mediante contenedores Podman (equivalentes a Docker), evaluando el impacto del volumen de información a diferentes escalas, 1,000, 10,000 y 100,000 registros por tabla. Asimismo, se analiza el procesamiento analítico de combinar la consulta desde bases de datos relacionales con motores de ejecución en memoria de alto rendimiento de Python, como lo es la librería Polars.

2. Esquema utilizado

En esta sección se presenta la parte del modelo Entidad-Relación utilizada para la implementación. Se eligieron de entre todas las entidades originales, solo 4 que estuvieran interconectadas.

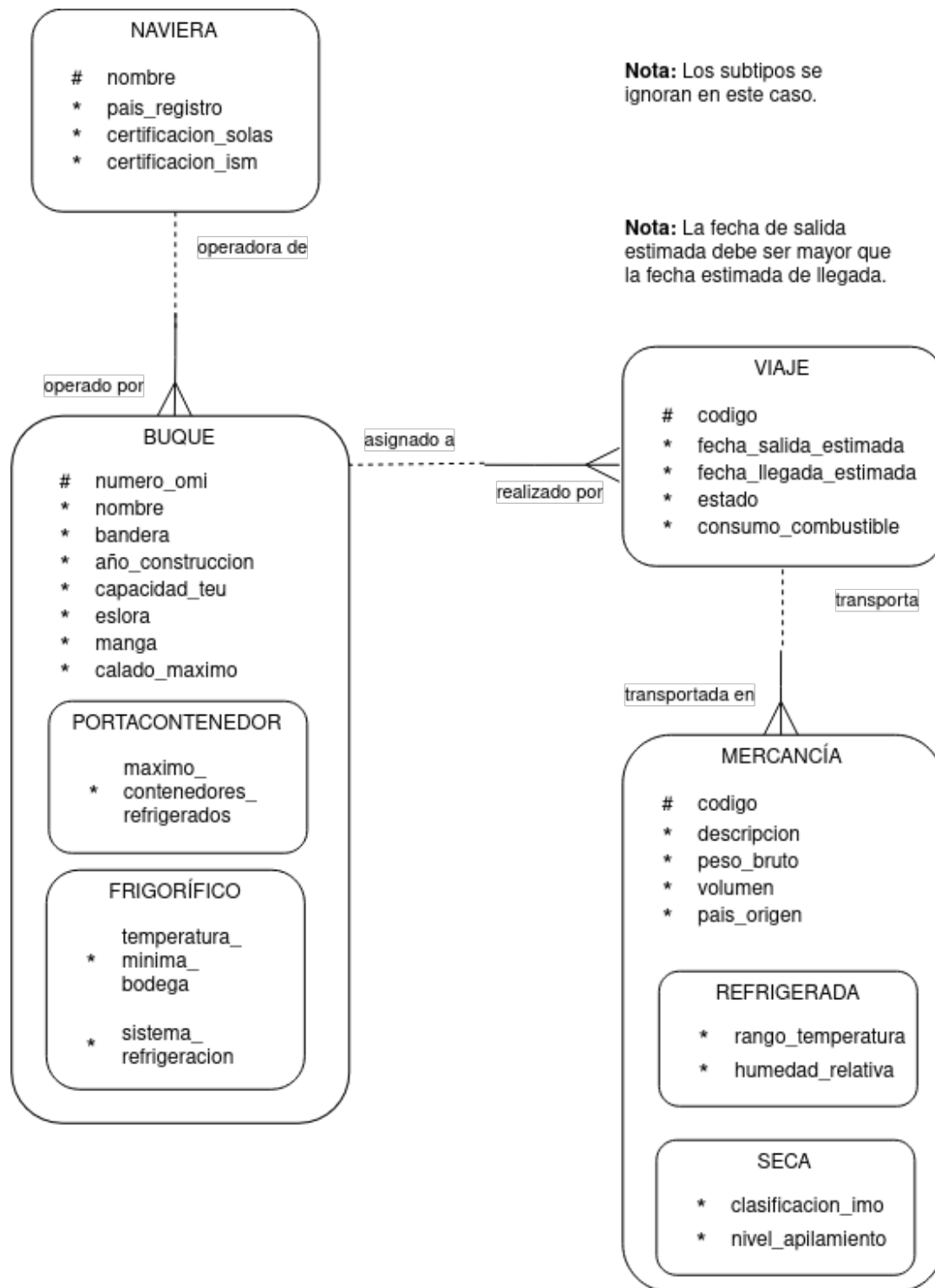


Figura 1: Fragmento del modelo E-R implementado.

3. Pruebas de carga

Se realizaron pruebas de inserción sobre ambos SGBD utilizando el mismo conjunto de datos. Tener en cuenta que para cada número de registros diferentes, se corrieron 10 pruebas, y se promediaron los resultados.

3.1. Resultados

Prueba	Número de registros	MySQL (s)	PostgreSQL (s)
1	1.000	0,235	0,084
2	10.000	1,941	0,424
3	100.000	7,303	4,297

Cuadro 1: Tiempo de inserción en cada SGBD.

Esta gráfica fue generada con ayuda de Inteligencia Artificial a partir del resultado en consola del programa.

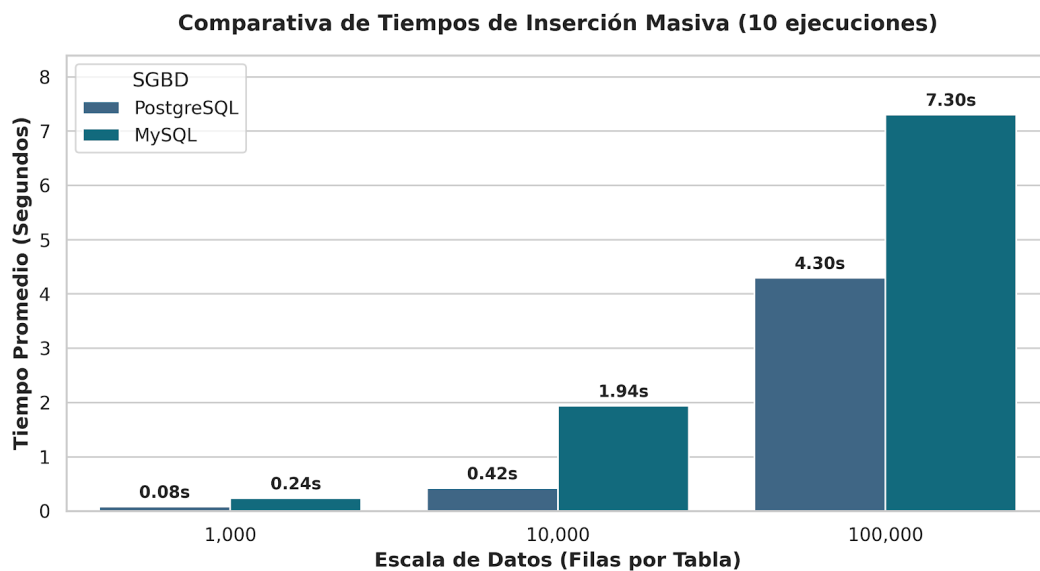


Figura 2: Comparación de tiempos de inserción.

4. Benchmark de consultas

Se evaluaron dos consultas utilizando tres enfoques:

- MySQL
- PostgreSQL
- Resolución mediante Python

4.1. Consulta 1

Descripción

Obtener el listado detallado de todos los viajes registrados en el sistema, mostrando el nombre de la naviera responsable, el nombre del buque asignado, junto con el código del viaje, su estado actual y su fecha estimada de salida.

Restricción

Esta consulta debe incluir JOINS con tres de las tablas del esquema implementado.

Fragmento representativo y equivalente de la consulta utilizada tanto en PostgreSQL como en MySQL.

Listing 1: Consulta 1.

```
1 SELECT N.nombre, B.nombre, V.codigo, V.estado, V.  
   fecha_estimada_salida  
2 FROM NAVIERA N  
3 JOIN BUQUE B ON N.nombre = B.nombre_naviera  
4 JOIN VIAJE V ON B.numero_omi = V.buque_numero_omi;
```

Implementación	1000 registros	10000 registros	100000 registros
MySQL	0.0090	0.0924	0.6388
PostgreSQL	0.0060	0.0282	0.2566
Polars (MySQL)	0.0217	0.0481	0.3318
Polars (Postgres)	0,0582	0,0962	0,5499

Cuadro 2: Tiempo promedio de ejecución de la Consulta 1 (segundos).

4.2. Consulta 2

Descripción

Calcular la cantidad total de viajes realizados agrupados por cada naviera y por el país de abanderamiento (bandera) del buque, presentando los resultados ordenados de mayor a menor según el total de viajes registrados.

Restricción

Esta consulta debe incluir un GROUP BY que involucra dos tablas y un JOIN con otra tabla. El resultado final debe ser ordenado mediante el operador ORDER BY.

Fragmento representativo y equivalente de la consulta utilizada tanto en PostgreSQL como en MySQL.

Listing 2: Consulta 2.

```
1 SELECT N.nombre AS naviera, B.bandera AS bandera, count(V.  
   codigo) AS total_viajes  
2 FROM NAVIERA N  
3 JOIN BUQUE B ON N.nombre = B.nombre_naviera  
4 JOIN VIAJE V ON B.numero_omi = V.buque_numero_omi  
5 GROUP BY N.nombre, B.bandera  
6 ORDER BY count(V.codigo) DESC;
```

Implementación	1000 registros	10000 registros	100000 registros
MySQL	0.0083	0.0548	0.5369
PostgreSQL	0.0060	0.0219	0.6060
Polars (MySQL)	0.0223	0.0501	0.3676
Polars (Postgres)	0,0582	0,0981	0,5760

Cuadro 3: Tiempo promedio de ejecución de la Consulta 2 (segundos).

Esta gráfica fue generada con ayuda de Inteligencia Artificial a partir del resultado en consola del programa.

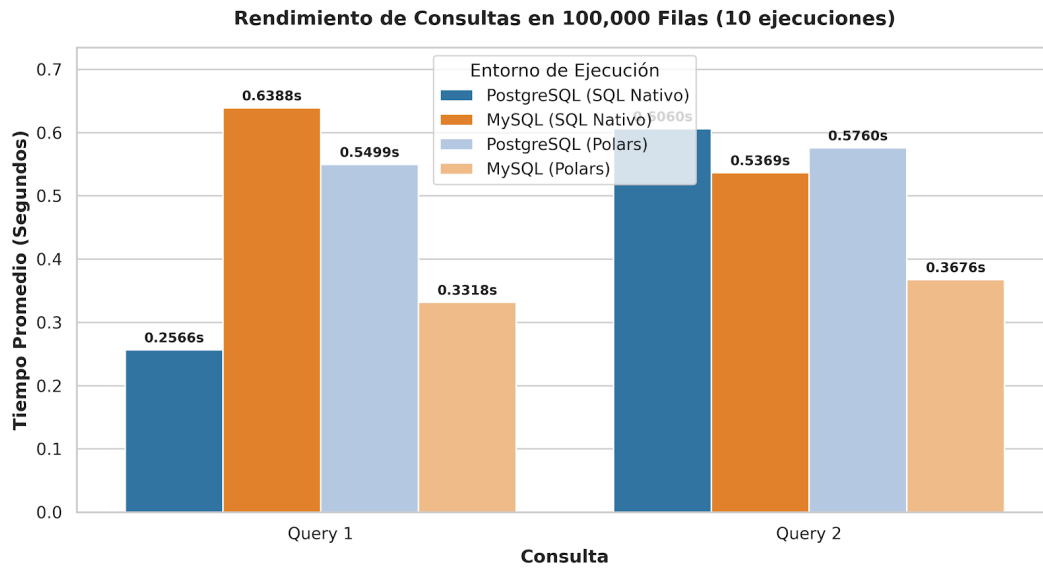


Figura 3: Comparación de tiempos de consulta.

5. Conclusiones

Los resultados obtenidos muestran diferencias claras en el desempeño de los dos sistemas gestores de bases de datos (SGBD) evaluados. Durante las pruebas de inserción, PostgreSQL presentó un mejor rendimiento en todos los tamaños de carga. Para 100 000 registros alcanzó un tiempo promedio de 4,30 s, mientras que MySQL requirió 7,30 s, lo que representa una diferencia de mayor velocidad en un 41 % para PostgreSQL. Esta diferencia también se mantuvo para el conjunto de 1 000 y 10 000 filas de información. Se puede evidenciar claramente una mayor eficiencia de PostgreSQL en operaciones masivas de escritura.

En las consultas SQL ejecutadas directamente sobre los SGBD, PostgreSQL obtuvo mejores tiempos en la Consulta 1, especialmente para el conjunto de 100 000 registros, donde registró un promedio de 0,257 s frente a 0,639 s de MySQL. En la Consulta 2, por otro lado, el comportamiento fue muy diferente: MySQL alcanzó un tiempo promedio de 0,537 s, superando por poco a PostgreSQL, que obtuvo 0,606 s. En este caso, los resultados apuntan a que MySQL es capaz de gestionar de mejor manera consultas más complejas, que intervengan múltiples JOINS, cláusulas y

funciones de agrupamiento. El rendimiento demuestra que igualmente, ambos están por decirlo así, a la par, sin una diferencia en tiempo tan significativa.

La implementación realizada en Python presentó tiempos considerablemente menores que las consultas SQL nativas una vez los datos fueron cargados en memoria. Para ambas consultas y en ambos motores, los tiempos se mantuvieron por debajo de 0,03 s incluso con 100 000 registros. Esta conducta se debe a la implementación de la librería especializada Polars. Esta librería, construida en Rust, aprovecha el paralelismo, la vectorización y el uso eficiente de la memoria RAM, evitando el costo asociado a la planificación y ejecución de consultas dentro del SGBD.

Cabe aclarar que existe un aspecto muy importante en la medición. Dada la naturaleza de la implementación y del experimento, este tiempo se tiene en cuenta desde la consulta `SELECT *` a todas las tablas de la base de datos, junto con el tiempo de procesamiento de Polars. El tiempo de la ejecución Polars con las consultas cuando los datos ya han sido extraídos, roza tiempos de 0,03 segundos en promedio según pruebas con una implementación alterna.

Origen de los datos	Consulta	1000 registros	10000 registros	100000 registros
PostgreSQL	Consulta 1	0.0582	0.0962	0.5499
PostgreSQL	Consulta 2	0.0582	0.0981	0.5760
MySQL	Consulta 1	0.0217	0.0481	0.3318
MySQL	Consulta 2	0.0223	0.0501	0.3676

Cuadro 4: Tiempo promedio de ejecución de las consultas implementadas con Polars según el SGBD desde el cual se cargaron los datos (segundos).

Podemos ver que la extracción de todas las filas en cada SGBD es lo que representa algo así como el tiempo base, y las pequeñas variaciones entre consulta 1 y 2, demuestran la rapidez de Polars.

En términos generales, PostgreSQL demostró ser la alternativa más eficiente para operaciones de inserción y para una de las consultas evaluadas, mientras que MySQL obtuvo un mejor desempeño en la segunda consulta SQL. Por otra parte, el procesamiento en memoria con Polars evidenció ser una herramienta muy eficiente para tareas de análisis una vez los datos se encuentran disponibles localmente, pero que no siempre supera las consultas de SQL nativo. Esto pone en evidencia la responsabilidad de cada persona en la elección de la tecnología más adecuada según su necesidad.